

# Trekking Management Application - V2 Project Report

## 1. Student Details

**Name:** Subhradeep Bhoemik

**Roll Number:** 23f3002696

**Email:** 23f3002696@ds.study.iitm.ac.in

**About Me:** As a person who enjoys exploring new ideas and concepts, I am always eager to learn more about the Technological World around me, and I'm not afraid to ask questions. I am also a very deep thinker. So, I often find myself contemplating the meaning of Life and the Universe with the eyes of Science and Technology.

## 2. Project Details

**Project Title:** Silver Fortnight

**Problem Statement:**

Manual trek management via spreadsheets and phone calls creates inefficiencies in slot tracking, staff assignment, and participant registration.

**Approach:**

Built a role-based web app (Vue 3, Flask, SQLite, Redis/Celery) centralizing workflows for Admins, Staff, and Trekkers, enforcing API-level logic to prevent overbooking and unauthorized access.

## 3. AI/LLM Declaration

I used Claude (Sonnet 4.6) to assist in creating charts for the dashboard (front-end), UI styling, and to debug. The extent of AI/LLM usage is around 15-20%, limited to code suggestions.

All final implementations including business logic, database relationships, and core task scheduling were implemented manually.

## 4. Technologies and Frameworks Used

| Technology / Library        | Purpose                                                    |
|-----------------------------|------------------------------------------------------------|
| Flask                       | Core backend REST API framework                            |
| Flask-SQLAlchemy            | Database ORM and object mapping                            |
| Flask-JWT-Extended          | JWT authentication and role-based access control (RBAC)    |
| Flask-CORS                  | Cross-Origin Resource Sharing for frontend API requests    |
| Flask-Caching (RedisCache)  | API response caching with fallback handling                |
| Celery + Celery Beat        | Asynchronous background tasks and scheduled jobs           |
| Redis                       | Message broker, result backend, and cache store            |
| SQLite                      | Primary relational database                                |
| Vue + Vue Router (CDN)      | Single-page frontend with hash-based, role-guarded routing |
| Axios                       | HTTP client for API requests                               |
| Bootstrap + Bootstrap Icons | Responsive UI layout, styling, and icons                   |
| Jinja2                      | HTML page entry point and email template rendering         |

## 5. Database Schema

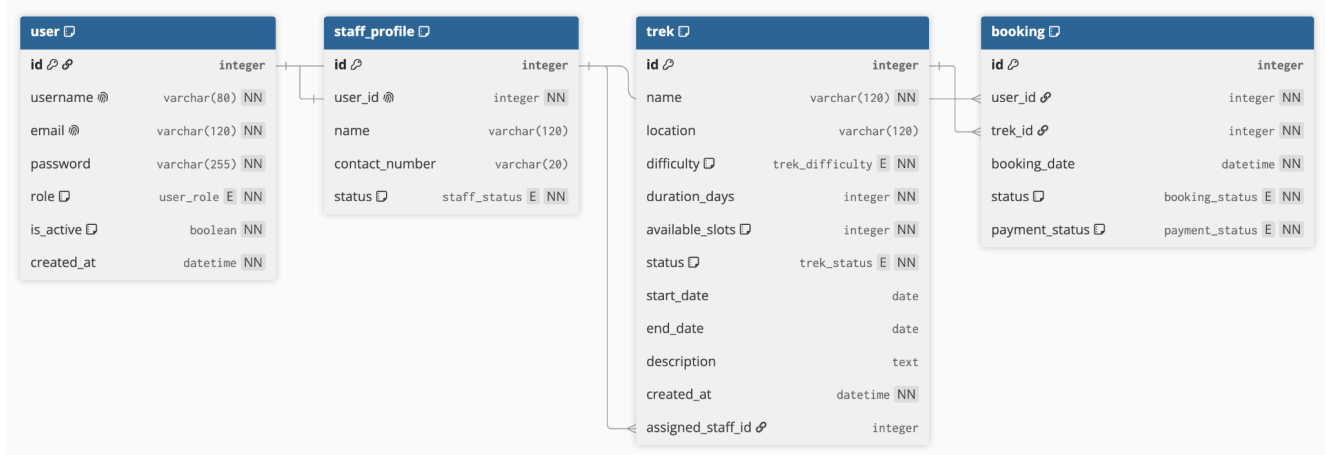
**Tables:**

- User** - id, username (unique), email (unique), password (hashed), role (admin / staff / user), is\_active, created\_at
- StaffProfile** - id, user\_id (FK, unique), name, contact\_number, status (active / deactivated)

- **Trek** - id, name, location, difficulty (easy / moderate / hard), duration\_days, available\_slots, status (pending / approved / open / closed / completed), start\_date, end\_date, assigned\_staff\_id (FK, nullable)
- **Booking** - id, user\_id (FK), trek\_id (FK), booking\_date, status (booked / cancelled / completed), payment\_status

#### Relationships:

- User ↔ StaffProfile — one-to-one (cascade delete)
- User ↔ Booking — one-to-many (cascade delete)
- StaffProfile ↔ Trek — one-to-many, via assigned\_staff\_id (nullable until Admin assigns staff)
- Trek ↔ Booking — one-to-many (cascade delete)



## 6. API Resource Endpoints

All endpoints are prefixed with /api and are JWT-protected except registration and login. Role-based access is enforced with a role\_required decorator.

| Group | Method       | Endpoint                            | Description                                       |
|-------|--------------|-------------------------------------|---------------------------------------------------|
| Auth  | POST         | /api/auth/register                  | Trekker self-registration                         |
| Auth  | POST         | /api/auth/login                     | Login for all roles; issues JWT                   |
| Auth  | POST         | /api/auth/logout                    | Revokes the current JWT (blocklist)               |
| Auth  | GET          | /api/auth/me                        | Returns the current authenticated user            |
| Admin | GET          | /api/admin/dashboard                | Trek / user / staff / booking counts (cached)     |
| Admin | GET / POST   | /api/admin/treks                    | List (cached, filterable) / create treks          |
| Admin | PUT / DELETE | /api/admin/treks/<id>               | Update / remove a trek                            |
| Admin | GET / POST   | /api/admin/staff                    | List (cached) / create trek staff                 |
| Admin | PUT / DELETE | /api/admin/staff/<id>               | Update / remove staff                             |
| Admin | POST         | /api/admin/staff/<id>/toggle-active | Activate / deactivate a staff member              |
| Admin | GET          | /api/admin/users                    | List trekkers (cached, searchable)                |
| Admin | POST         | /api/admin/users/<id>/toggle-active | Blacklist / reactivate a trekker                  |
| Admin | GET          | /api/admin/bookings                 | View all booking records                          |
| Admin | POST         | /api/admin/reports/monthly/run      | Manually trigger the monthly activity report job  |
| Staff | GET          | /api/staff/dashboard                | Assigned treks and registered-trekker counts      |
| Staff | GET          | /api/staff/treks                    | List treks assigned to the logged-in staff member |

|       |      |                                      |                                                      |
|-------|------|--------------------------------------|------------------------------------------------------|
| Staff | PUT  | /api/staff/treks/<id>                | Update slots / status / completion for an owned trek |
| Staff | GET  | /api/staff/treks/<id>/participants   | List participants for an owned trek                  |
| Staff | POST | .../participants/<booking_id>/cancel | Cancel a participant's booking                       |
| User  | GET  | /api/user/dashboard                  | Trekker's dashboard summary                          |
| User  | GET  | /api/user/treks                      | Browse / search / filter open treks (cached)         |
| User  | POST | /api/user/treks/<id>/book            | Book a trek (blocks duplicates & overbooking)        |
| User  | GET  | /api/user/bookings                   | View own booking / trekking history                  |
| User  | POST | /api/user/bookings/<id>/cancel       | Cancel own booking                                   |
| User  | PUT  | /api/user/profile                    | Update own profile                                   |
| User  | POST | /api/user/bookings/export            | Trigger async CSV export of booking history          |
| User  | GET  | .../export/<task_id>                 | Poll export job status                               |
| User  | GET  | .../export/<task_id>/download        | Download the completed CSV                           |

## 7. Architecture and Features

### Architecture Overview:

- backend/app.py — Flask application factory, CLI commands (init-db, create-admin, create-staff), and Celery periodic-task registration (daily reminders at 08:00, monthly report on the 1st).
- backend/application/ — models.py (SQLAlchemy models), constants.py (role/status enums), security.py (JWT loaders), cache.py (Flask-Caching query builders + invalidation), routes.py (all REST endpoints), tasks.py (Celery tasks), mail.py (SMTP sender), celery\_init.py / celery\_config.py.
- frontend/templates/index.html — single Jinja2 entry point that loads Vue, Vue Router, Axios and Bootstrap from CDN.
- frontend/static/js/ — app.js (root Vue component), router.js (role-guarded routes), auth.js (auth state), api.js (Axios instance), and views/ split by role (admin/, user/, plus Login, Register, StaffDashboard).

### Implemented Features:

- JWT authentication with role-based access control; self-registration for Trekkers, Admin/Staff created only by Admin (or CLI).
- Admin: dashboard stats, full trek CRUD, staff CRUD with activate/deactivate, user search and blacklist, view of all bookings, manual monthly-report trigger.
- Trek Staff: dashboard of assigned treks, slot/status updates, participant list management — restricted to their own assigned treks.
- Trekkers: search/filter open treks, book/cancel with duplicate- and overbooking-prevention, dashboard and booking history, profile updates.
- Backend jobs (Celery + Redis): daily trek reminder emails, monthly HTML activity report to Admins with popular-trek stats, and user-triggered async CSV export of booking history with status polling and a completion email.
- Performance: Redis-backed caching of trek listings, admin dashboard stats, and staff/user lists, invalidated on writes, with automatic fallback to an in-process cache if Redis is unavailable.

### Additional Features:

- Graceful Redis-outage fallback for caching (SimpleCache), rather than failing requests.
- Asynchronous CSV export flow (trigger → poll status → download) instead of a blocking export request.
- Admin-triggered on-demand run of the monthly report job, in addition to the scheduled run.

## 8. Video Presentation

**Drive Link:** <https://drive.google.com/file/d/1VGwNE19dJEN99h1voTs7uzHrDuWYdaOg/view>